Học viện
Công nghệ Bưu chính Viễn thông

PHÂN TÍCH DỮ LIỆU CHUỖI THỜI GIAN

Kỹ thuật xử lý dữ liệu chuỗi thời gian

Nội dung

- Nhãn thời gian
- Làm sạch dữ liệu - Khử nhiễu và làm mịn - Tăng cường
- Mô phỏng dữ liệu chuỗi thời gian
- Lưu trữ dữ liệu chuỗi thời gian
- Trực quan hóa dữ liệu chuỗi thời gian

Nhãn thời gian

- Hiểu khái niệm nhãn thời gian (timestamp)
- Tại sao nhãn thời gian quan trọng
- Các dạng nhãn thời gian
- Một số lưu ý khi xử lý

Nhãn thời gian

➢ **Nhãn thời gian là gì?**

- Một giá trị hoặc tập giá trị biểu diễn **thời điểm xảy ra của một quan sát** trong chuỗi dữ liệu.
- Ví dụ:

Thời gian	Nhiệt độ
2025-07-22 09:00	30°C
2025-07-22 10:00	31°C

Ở đây, cột *Thời gian* chính là **nhãn thời gian**.

Nhãn thời gian

- **Các dạng nhãn thời gian**

📌 **Theo thời gian thực (Real-time)**

Ví dụ: 2025-07-22 09:15:23

📌 **Theo khoảng thời gian (Interval)**

Ví dụ: Tháng 1/2025, Tuần 23

📌 **Dạng số thứ tự (Index-based)**

Ví dụ: quan sát thứ 1, 2, 3 (nếu thời gian đều đặn)

Nhãn thời gian

➢ Tầm quan trọng của nhãn thời gian

- Xác định thứ tự và khoảng cách giữa các quan sát
- Phân tích xu hướng & mùa vụ
- Tính toán độ trễ, hồi quy, dự báo
- Gắn kết dữ liệu từ nhiều nguồn theo cùng trục thời gian

Nhãn thời gian

❑ Lưu ý

- Nhãn thời gian là yếu tố **bắt buộc** trong dữ liệu chuỗi thời gian
  - Giúp duy trì thứ tự & ý nghĩa dữ liệu
  - Cần xử lý cẩn thận để tránh sai sót khi phân tích
- Cần thận trọng với độ phân giải của nhãn thời gian nhận được, dựa trên:
  - kiến thức chuyên môn về dữ liệu đang được quan sát
  - các chi tiết có thể xác định liên quan đến cách dữ liệu được thu thập.

Nhãn thời gian

❑ Lưu ý

- Ví dụ:

Nhãn thời gian	Doanh thu theo ngày
2025-07-22	12
2025-07-23	18
2025-07-24	14

  - Thời gian cách đều 1 ngày
  - Độ phân giải: ngày

Nhãn thời gian

❑ Lưu ý

- Ví dụ:

Nhãn thời gian	Doanh thu theo ngày
2025-07-22	12
2025-07-29	18
2025-08-05	14

  - Thời gian cách đều 1 tuần
  - Độ phân giải: tuần

Nhãn thời gian

❑ Lưu ý

- Ví dụ:

Nhãn thời gian	Giá cổ phiếu
2025-07-22 09:00:00	102.5
2025-07-22 09:01:00	102.7
2025-07-22 09:02:00	102.6

  - Thời gian cách đều 1 phút
  - Độ phân giải: phút
  - Thích hợp cho phân tích biến động ngắn hạn

Nhãn thời gian

❑ Lưu ý

- Kiểm tra định dạng & chuẩn hóa
  - Ví dụ: YYYY-MM-DD HH:MM:SS
- Kiểm tra tính đồng nhất (khoảng cách giữa các mốc)
- **Xử lý dữ liệu thiếu / lỗi nhãn**
- Chuyển múi giờ (time zone) khi cần
- Lựa chọn độ phân giải phù hợp với mục tiêu phân tích.

🕒 Các mức độ thường gặp:

Giây (second) Phút (minute) Giờ (hour)

Ngày (day) Tuần / Tháng / Quý / Năm

Làm sạch dữ liệu

➢ Làm sạch dữ liệu (Data Cleaning) là gì?

Loại bỏ hoặc chỉnh sửa dữ liệu sai, thiếu, không hợp lệ

- **Khử nhiễu & làm mịn (Denoising & Smoothing)**
  Giảm bớt dao động ngẫu nhiên, làm rõ xu hướng hoặc mẫu
- **Tăng cường dữ liệu (Data Augmentation)**
  Tạo thêm dữ liệu để cải thiện mô hình học máy, đặc biệt khi dữ liệu ít

Làm sạch dữ liệu

- **Làm sạch dữ liệu (Data Cleaning) là gì?**

  Làm sạch giúp dữ liệu **đáng tin cậy hơn**
  - Khử nhiễu & làm mịn giúp **phát hiện xu hướng rõ ràng hơn**
  - Tăng cường giúp mô hình **học tốt hơn & tổng quát hơn**

Làm sạch dữ liệu

➢ Làm sạch dữ liệu (Data Cleaning) là gì

- Ví dụ:

Thời gian	Giá trị
2025-07-22 09:00	102
2025-07-22 09:01	NaN
2025-07-22 09:02	9999*

➞ Xử lý:

- Điền giá trị thiếu ở 09:01
- Loại bỏ/điều chỉnh 9999 (ngoại lai)

Làm sạch dữ liệu

➢ Mục đích của làm sạch dữ liệu

Khắc phục các vấn đề

- Thiếu dữ liệu (missing values)
- Thay đổi tần suất của chuỗi thời gian (tức là tăng và giảm mẫu)
- Giá trị ngoại lai (outliers)
- Sai định dạng thời gian
- Tính thời vụ trong dữ liệu
- Vấn đề nhìn trước (lookahead)

Làm sạch dữ liệu

- **Các kĩ thuật**
  - Điền giá trị thiếu (mean, median, interpolation)
  - Khử nhiễu và làm mịn dữ liệu (denoising & smoothing)
  - Thay đổi tần suất của chuỗi
  - Loại bỏ hoặc thay thế ngoại lai
  - Chuẩn hoá định dạng thời gian
  - Xử lý tính thời vụ trong dữ liệu
  - Ngăn ngừa việc nhìn trước không chủ ý

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Dữ liệu bị thiếu xảy ra khá phổ biến trong phân tích chuỗi thời gian do
    - Gánh nặng của việc lấy mẫu
    - Phép đo không chính xác (chủ quan/khách quan)
    - Nhập sai dữ liệu

Ví dụ: trong lĩnh vực chăm sóc sức khỏe, việc thiếu dữ liệu trong chuỗi thời gian y tế có thể xuất phát do bệnh nhân không tuân thủ đúng quy trình đo do quên hoặc làm không đầy đủ, tình trạng sức khỏe của bệnh nhân tốt nên bỏ qua phép đo, thiết bị y tế gặp trục trặc kỹ thuật ngẫu nhiên.

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - **Các hướng tiếp cận**
    - Quy nạp: điền dữ liệu bị thiếu dựa trên các quan sát về toàn bộ tập dữ liệu.
    - Nội suy: sử dụng các điểm dữ liệu lân cận để ước tính giá trị bị thiếu. Nội suy cũng có thể là một hình thức quy nạp.
    - Xóa các khoảng thời gian bị ảnh hưởng: không sử dụng các khoảng thời gian bị thiếu dữ liệu.

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - Điền tiếp (Forward fill)
    - Trung bình động (Moving average)
    - Nội suy (Interpolation)

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Điền tiếp** (Forward fill) là phương pháp thay thế giá trị bị thiếu bằng **giá trị gần nhất trước đó** trong chuỗi thời gian.
      - Một trong những cách đơn giản nhất để điền vào các giá trị bị thiếu
      - Chuyển tiếp giá trị đã biết cuối cùng trước giá trị bị thiếu.
      - Không cần toán học hay logic phức tạp.

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Điền tiếp** (Forward fill)

Giả sử chuỗi:

$$x_1, x_2, \dots, x_t (= NaN), x_{t+1}, \dots$$

Nếu x_t bị thiếu:

$$x_t = x_{t-1}$$

Nếu nhiều điểm liên tiếp bị thiếu:

$$x_t, x_{t+1}, \dots, x_{t+k} = x_{t-1}$$

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Điền tiếp** (Forward fill)

Ví dụ:

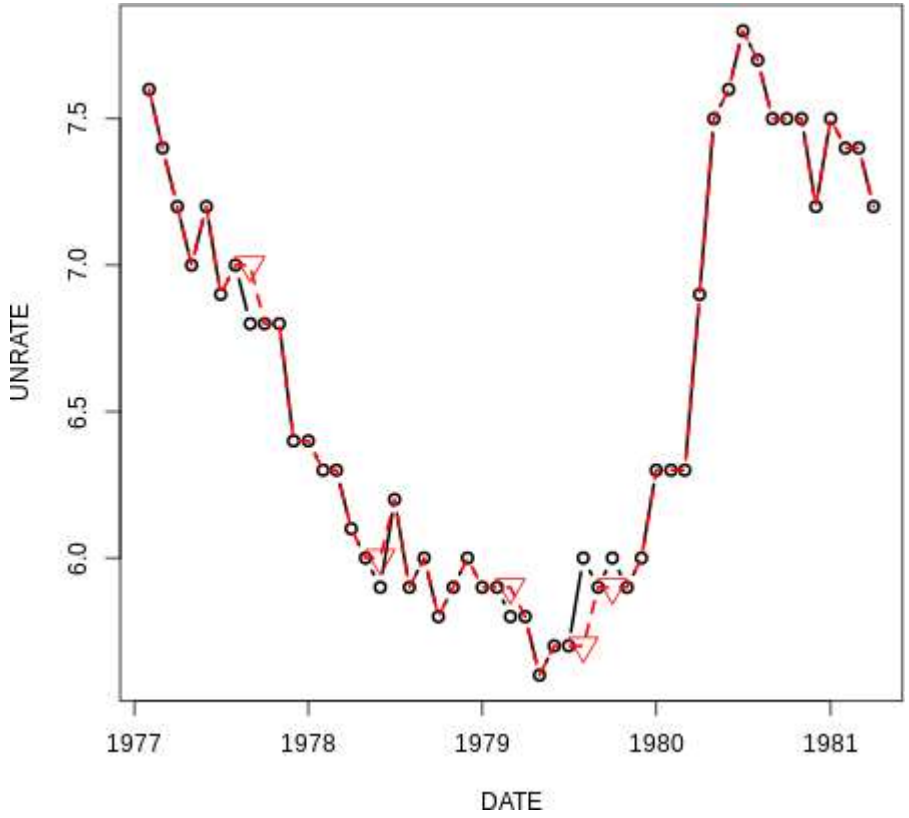
t	1	2	3	4
x_t	10	NaN	NaN	15

→ Forward fill:

t	1	2	3	4
x_t	10	**10**	**10**	15

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Điền tiếp** (Forward fill)



Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Điền tiếp** (Forward fill) phù hợp với những chuỗi mà:
      - Giá trị được giữ **ổn định** cho đến khi có thay đổi mới
      - Đặc trưng kiểu dữ liệu **step function** / piecewise constant
      - Ví dụ thực tế:
        - ✓ Giá trị tài khoản ngân hàng giữa các giao dịch
        - ✓ Trạng thái thiết bị (on/off)
        - ✓ Nhãn phân loại theo thời gian
        - ✓ Dữ liệu cảm biến bị mất nhưng thông số biến đổi chậm

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Điền tiếp** (Forward fill) có hạn chế
      - ✕ Không hợp khi giá trị thật thay đổi theo thời gian (trend mạnh)
        → forward fill "kéo dài" giá trị cũ → gây sai lệch lớn
      - ✕ Khoảng missing dài → dữ liệu bị phẳng (flat-line)
      - ✕ Không dùng được cho điểm thiếu ở vị trí đầu chuỗi
        → vì không có giá trị trước đó

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Trung bình động** (Moving average) là kỹ thuật làm mượt chuỗi thời gian bằng cách lấy trung bình của một số điểm lân cận.
      - Trung bình động đơn giản
      - Trung bình động hai phía
      - Trung bình động có trọng số

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Trung bình động đơn giản** (Simple Moving Average – SMA)
      - Giả sử chuỗi thời gian: $x_1, x_2, \ldots, x_t, \ldots, x_n$

        Nếu x_t bị thiếu, thay bằng

$$\hat{x}_t = \frac{1}{k}(x_{t-1} + x_{t-2} + \ldots + x_{t-k})$$

      - Có nhiều trường hợp mà việc quy nạp dữ liệu bằng trung bình động phù hợp hơn so với phương pháp điền trước.

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Trung bình động động đơn giản**
      - Ví dụ

t	1	2	3	4	5
x_t	10	12	NaN	13	14

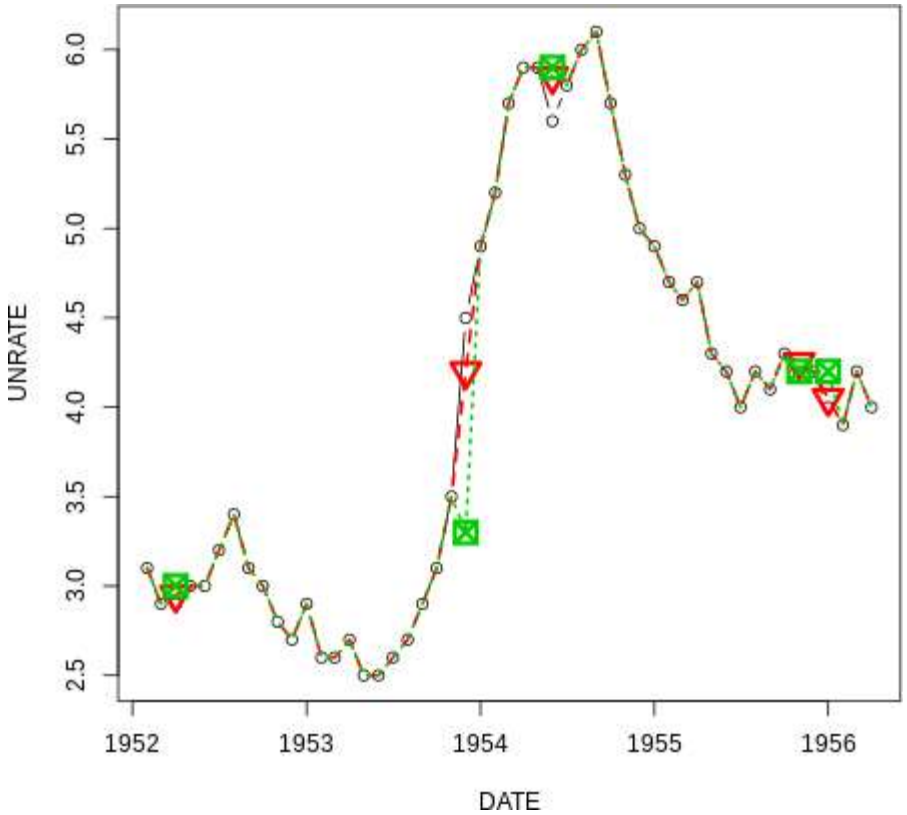
Dùng SMA với cửa sổ $k = 2$: $\hat{x}_3 = \frac{12+10}{2} = 11$

Kết quả:

t	1	2	3 (điền)	4	5
x_t	10	12	**11**	13	14

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Trung bình động đơn giản**
      - Ví dụ



Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Trung bình động hai phía (CMA)**
      Giả sử chuỗi thời gian: $x_1, x_2, \ldots, x_t, \ldots, x_n$
      Nếu x_t bị thiếu, thay bằng **trung bình động hai phía (Centered Moving Average – CMA)** khi điểm thiếu nằm giữa chuỗi:

$$\hat{x}_t = \frac{1}{2k}\left(\sum_{i=1}^{k} x_{t-i} + \sum_{i=1}^{k} x_{t+i}\right)$$

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Trung bình động có trọng số (WMA)**

      Giả sử chuỗi thời gian: $x_1, x_2, \ldots, x_t, \ldots, x_n$

      Nếu x_t bị thiếu, thay bằng **trung bình động có trọng số (Weighted Moving Average - WMA)**

      Ưu tiên gần → điểm gần có trọng số lớn hơn.

$$\hat{x}_t = \sum_{i=1}^{k} w_i \, x_{t-i}, \qquad \sum w_i = 1$$

      Ví dụ: trọng số giảm dần theo khoảng cách.

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Trung bình động** (Moving average) nên dùng khi
      - Dữ liệu thiếu ít (1–2 điểm)
      - Chuỗi mượt và không có nhiễu lớn
      - Không có thay đổi đột ngột
      - Cần một phương pháp nhanh, đơn giản, ít giả định
    - Hạn chế
      - Làm **mất tính biến động tự nhiên** khi dữ liệu có dao động mạnh.
      - Không phù hợp nếu có **nhiều điểm liên tiếp bị thiếu**.
      - Dữ liệu theo mùa vụ (seasonality) có thể bị làm phẳng quá mức.

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Nội suy** (Interpolation) là phương pháp ước lượng giá trị bị thiếu dựa trên xu hướng của *các điểm dữ liệu lân cận*.
    - Các cách nội suy:
      - Nội suy tuyến tính: chia đều toàn bộ khoảng của nhiều điểm thiếu liên tiếp. Nên dùng khi xu hướng thay đổi đều, không có biên độ biến thiên đột ngột.
      - Nội suy bậc cao (bậc ba là phổ biến nhất) cố gắng giữ độ cong của đường tại các đoạn. Giúp giữ được hình dạng tín hiệu tốt hơn (Mượt hơn tuyến tính - Không gây gãy khúc).

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Nội suy tuyến tính** (Linear Interpolation) – đơn giản và thông dụng nhất
      - Ý tưởng: giá trị tại thời điểm bị thiếu là điểm nằm trên đoạn thẳng nối trước và sau. Giả sử:

        Tại thời điểm t_1 , giá trị là x_1

        Tại thời điểm t_2 , giá trị là x_2

        Nếu x_t thiếu với t nằm giữa $t_1$và t_2, sử dụng ước lượng

$$\hat{x}(t) = x_1 + \frac{(t - t_1)}{(t_2 - t_1)}(x_2 - x_1)$$

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Nội suy tuyến tính** (Linear Interpolation)
      - Ví dụ

t	1	2	3	4	5
x	10	NaN	NaN	16	20

Đoạn mất từ t = 2 đến t = 3.

Hai điểm biên $(t_1 = 1, x_1 = 10)$, $(t_2 = 4, x_2 = 16)$

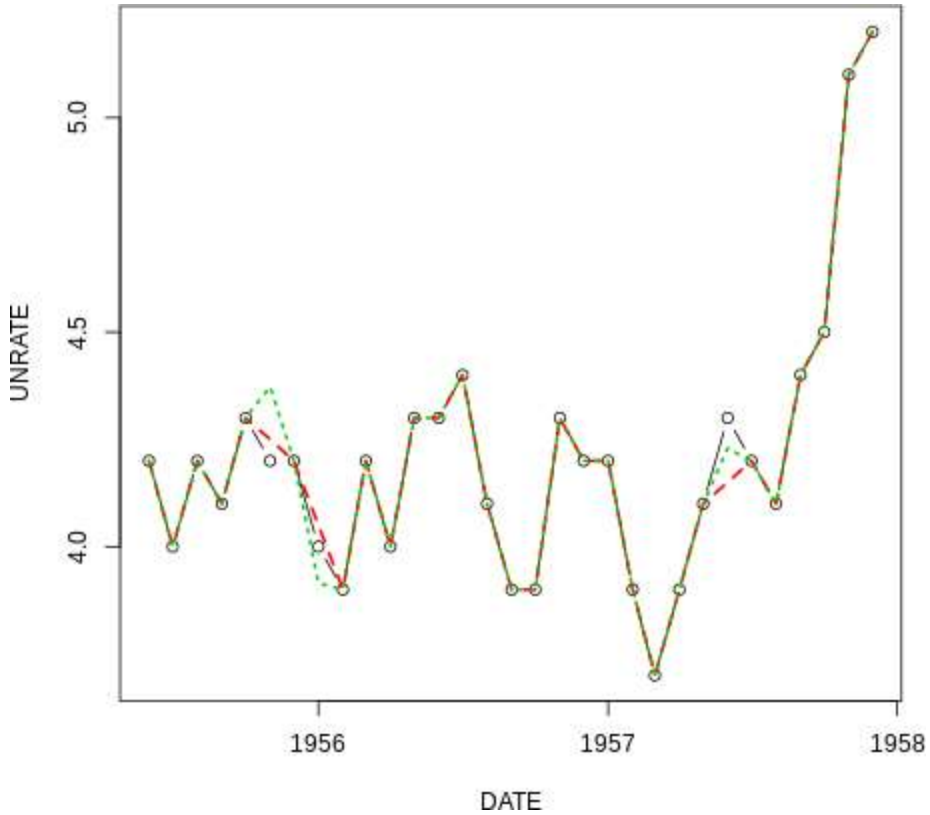
Khoảng cách: $t_2 - t_1 = 3$

$$x_2 = 10 + \frac{1}{3}(16 - 10) = 10 + 2 = 12$$

$$x_3 = 10 + \frac{2}{3}(16 - 10) = 10 + 4 = 14$$

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Nội suy tuyến tính** (Linear Interpolation)
      - Ví dụ



Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Nội suy bậc cao** (Spline Interpolation) là phương pháp nội suy dùng các **đa thức bậc thấp (thường là bậc 3 – cubic)** ghép lại với nhau ở từng đoạn
      - Ở **mỗi đoạn giữa hai điểm**, xài *một đa thức bậc 3 riêng*
      - Các đa thức ghép lại **liên tục** ở điểm nối (knot)
      - Đảm bảo liên tục cả:
        - ✓ hàm số
        - ✓ đạo hàm bậc 1
        - ✓ đạo hàm bậc 2

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Nội suy bậc ba** (cubic spline Interpolation)
      - Giả sử hai điểm liên tiếp: (t_i, x_i), (t_{i+1}, x_{i+1})
      - Spline bậc 3 trên đoạn này có dạng:

$$S_i(t) = a_i + b_i(t - t_i) + c_i(t - t_i)^2 + d_i(t - t_i)^3$$

      - Có **4 hệ số** cho mỗi đoạn → các hệ số được tính sao cho:

$$S_i(t_i) = x_i$$
$$S_i(t_{i+1}) = x_{i+1}$$
$$S_i'(t_{i+1}) = S_{i+1}'(t_{i+1})$$
$$S_i''(t_{i+1}) = S_{i+1}''(t_{i+1})$$

      - Nghĩa là: giá trị, đạo hàm và độ cong đều nối "khớp" tại điểm nối.

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Nội suy bậc ba** (cubic spline Interpolation)
      - Ví dụ

t	1	2	3	4	5
x	10	NaN	NaN	16	20

Kết quả

t	1	2	3	4	5
x	10	12.1	14.3	16	20

Làm sạch dữ liệu

- **Xử lý dữ liệu bị thiếu** (Handling Missing Data)
  - Một số cách cụ thể để điền số vào các giá trị còn thiếu:
    - **Nội suy** (Interpolation) nên dùng khi:
      - Dữ liệu bị thiếu **rải rác**.
      - Chuỗi có xu hướng rõ ràng và không nhiễu quá mạnh.
      - Cần giữ biên dạng tín hiệu thay vì làm mượt (khác moving average).
    - Không thích hợp khi:
      - Dữ liệu biến động mạnh, nhiễu lớn.
      - Có nhiều điểm liên tiếp bị thiếu (khoảng trống dài).
      - Dữ liệu có tính mùa vụ (seasonality) → nội suy có thể phá hỏng mô hình.

Làm sạch dữ liệu

- **Thay đổi tần suất của chuỗi** (Upsampling and Downsampling)
  - Nguyên nhân cần thay đổi tần suất
    - Thông thường, dữ liệu chuỗi thời gian liên quan từ các nguồn khác nhau sẽ không có cùng tần suất lấy mẫu. Đây là một trong nhiều lý do chính cho việc đòi hỏi thay đổi tần suất lấy mẫu của dữ liệu.
    - Chúng ta không thể thay đổi tốc độ thực tế mà thông tin được đo lường, nhưng có thể thay đổi tần suất của các nhãn thời gian trong bộ sưu tập dữ liệu của mình. Điều này được gọi là tăng tần suất lấy mẫu và giảm tần suất lấy mẫu, tương ứng với việc tăng hoặc giảm tần suất nhãn thời gian. Đây là hai thao tác cơ bản để đổi "độ phân giải thời gian" của dữ liệu.

Làm sạch dữ liệu

- **Thay đổi tần suất của chuỗi** (Upsampling and Downsampling)
  - **Giảm tần suất** (Downsampling - giảm độ phân giải)
    Chuyển chuỗi dữ liệu từ **tần suất cao → tần suất thấp**.
    - Chúng ta "gom" nhiều điểm nhỏ thành một điểm lớn.
      Ví dụ:Từ dữ liệu 1 phút → 15 phút, 1 giây → 1 phút, daily → monthly
    - Tại sao cần downsample?
      - Giảm nhiễu
      - Nhìn xu hướng tổng quát
      - Giảm kích thước dữ liệu
      - Chuẩn bị dữ liệu cho mô hình chỉ chấp nhận tần suất thấp hơn

Làm sạch dữ liệu

- **Thay đổi tần suất của chuỗi** (Upsampling and Downsampling)
  - **Giảm tần suất** (Downsampling - giảm độ phân giải)
    - Các giá trị thường dùng:
      - mean (trung bình): làm mượt dữ liệu → giảm nhiễu.
      - sum (tổng): dùng khi giá trị mang ý nghĩa tích lũy
      - min / max: giúp bắt được cực trị trong từng khoảng.
      - Median: hiệu quả trong dữ liệu nhiễu hoặc có outlier.

Làm sạch dữ liệu

- **Thay đổi tần suất của chuỗi** (Upsampling and Downsampling)
  - **Giảm tần suất** (Downsampling - giảm độ phân giải)
    - Ví dụ

Time	Value
10:00	5
10:01	7
10:02	6
10:03	9
10:04	11
10:05	10
10:06	8
10:07	12
10:08	15
10:09	14

Gom theo các đoạn:

Window 1: 10:00 → 10:04

→ values = [5, 7, 6, 9, 11]

Window 2: 10:05 → 10:09

→ values = [10, 8, 12, 15, 14]

Bảng tổng hợp:

Time	Mean	Sum	Max	Min	Median
10:00	7.6	38	11	5	7
10:05	11.8	59	15	8	12

Làm sạch dữ liệu

- **Thay đổi tần suất của chuỗi** (Upsampling and Downsampling)
  - **Tăng tần suất** (Upsampling - tăng độ phân giải)
    - Chuyển chuỗi dữ liệu từ tần suất thấp → tần suất cao.
    - Ví dụ: Từ dữ liệu 1 giờ → 1 phút, daily → hourly
    - Upsampling tạo ra **nhiều điểm bị thiếu (NaN)** → vì không có dữ liệu thật tại những thời điểm mới. → Sau khi tạo thêm thời điểm mới, ta thường phải **điền dữ liệu (filling)**:
      - Forward filling
      - Backward filling: khác với forward filling (dùng dữ liệu của nhãn thời gian gần nhất phía trước) thì backward filling dùng dữ liệu nhãn thời gian ở phía sau.
      - Nội suy: linear / spline

Làm sạch dữ liệu

➢ Khử nhiễu & làm mịn (Denoising & Smoothing)

- **Làm mịn** là kỹ thuật giảm nhiễu (noise) trong chuỗi thời gian bằng cách thay thế giá trị gốc bằng giá trị "mượt hơn" được tính từ các điểm lân cận.
- Mục đích : Giảm bớt dao động ngẫu nhiên, làm rõ xu hướng hoặc mẫu. **Làm mịn giúp:**
  - Dễ nhìn xu hướng
  - Hỗ trợ dự đoán (forecasting)
  - Chuẩn hóa đầu vào cho mô hình ML
  - Phát hiện bất thường hiệu quả hơn
  - Giảm nhiễu trong dữ liệu cảm biến

Làm sạch dữ liệu

➢ Khử nhiễu & làm mịn (Denoising & Smoothing)

- Nguyên nhân nhiễu:
  - Nhiễu đo lường. Ví dụ: Cảm biến lỗi
  - Dao động ngẫu nhiên. Ví dụ: Yếu tố ngẫu nhiên trong môi trường
  - Mất mát dữ liệu gây giật khúc
  - Giá trị ngoại lai (outlier)
- Các kĩ thuật
  - Moving Average
  - Exponential Smoothing
  - Kalman filter
  - Wavelet Denoising

Làm sạch – Khử nhiễu & Làm mịn – Tăng cường dữ liệu

➢ Khử nhiễu & làm mịn (Denoising & Smoothing)

- Các kĩ thuật làm mịn phổ biến
  - **Moving Average (trung bình trượt)** – đơn giản nhất

$$\hat{x}_t = \frac{1}{k}(x_t + x_{t-1} + \ldots + x_{t-k+1})$$

Mỗi điểm mới = trung bình của k điểm gần nhất.

- Ưu điểm:
  - ✓ Dễ dùng
  - ✓ Hiệu quả với nhiễu nhỏ
  - ✓ Không gây lệch pha nếu dùng trung bình đối xứng
- Nhược điểm:
  - ✗ Làm mất chi tiết
  - ✗ Xóa đỉnh/nhiều dao động thật
  - ✗ Trễ tín hiệu nếu dùng average về phía quá khứ

Làm sạch – Khử nhiễu & Làm mịn – Tăng cường dữ liệu

➢ Khử nhiễu & làm mịn (Denoising & Smoothing)

- Các kĩ thuật làm mịn phổ biến
  - **Weighted Moving Average** (WMA – trung bình trượt có trọng số)
    - Các điểm gần hiện tại được gán trọng số cao hơn.

$$\hat{x}_t = \frac{w_1 x_t + w_2 x_{t-1} + \ldots + w_k x_{t-k+1}}{\sum w_i}$$

    - Gaussian Smoothing: dùng trọng số theo phân phối chuẩn:

$$w_i = e^{-\frac{i^2}{2\sigma^2}}$$

      ✓ Làm mượt mềm mại
      ✓ Giữ đặc trưng biên tốt hơn MA

Làm sạch – Khử nhiễu & Làm mịn – Tăng cường dữ liệu

➢ Khử nhiễu & làm mịn (Denoising & Smoothing)

- Các kĩ thuật làm mịn phổ biến
  - **Exponential Moving Average (EMA)**

$$\hat{x}_t = \alpha x_t + (1-\alpha)\hat{x}_{t-1}$$

    - Trong đó: hệ số làm mịn **α (alpha)** luôn nằm trong khoảng:

$$0 < \alpha \leq 1$$

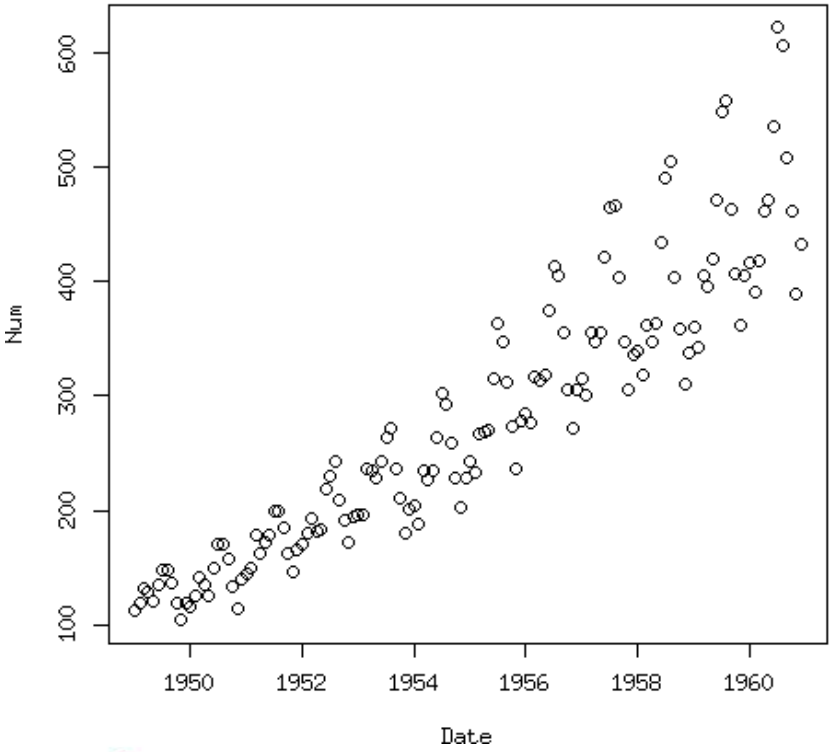
      ✓ α càng lớn → bám dữ liệu gốc
      ✓ α nhỏ → làm mượt mạnh hơn
    - EMA phản ứng nhanh hơn MA, phù hợp cho dữ liệu có xu hướng thay đổi. (phổ biến trong tài chính, cảm biến)

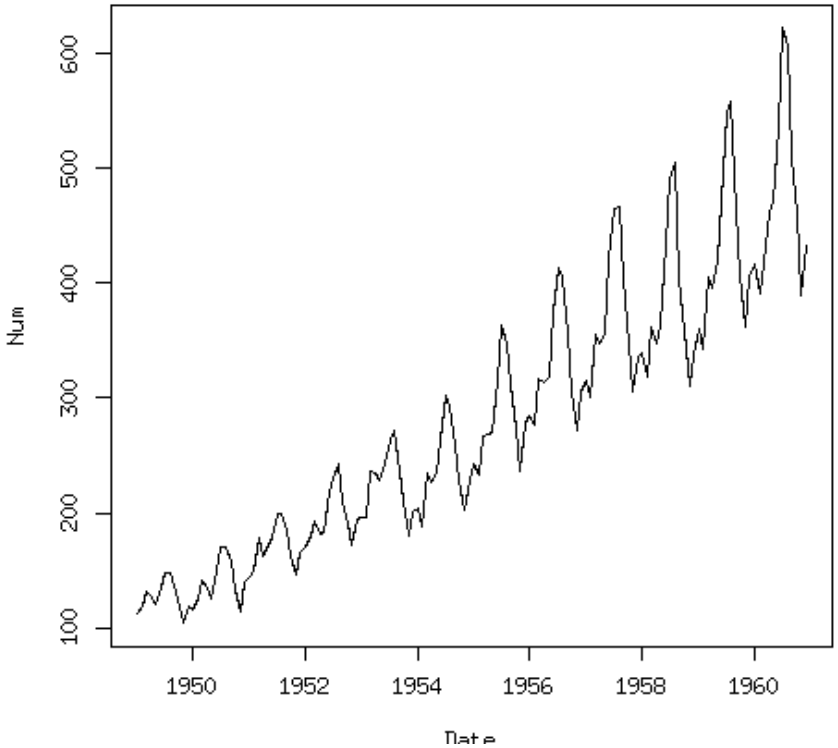
Làm sạch dữ liệu

- **Xử lý tính thời vụ trong dữ liệu**
  - Tính thời vụ trong dữ liệu là bất kỳ loại hành vi lặp lại nào mà tần suất của hành vi đó là ổn định. Nó có thể xảy ra ở nhiều tần suất khác nhau cùng một lúc.
    - Ví dụ: ăn trưa cùng giờ mỗi ngày, ngày đầu năm mới có lưu lượng giao thông thấp, chu kỳ Trái Đất quay quanh Mặt Trời
  - Việc xác định và xử lý tính thời vụ là một phần của quá trình lập mô hình.
  - Mặt khác, nó cũng là một hình thức làm sạch dữ liệu: **"tách" hoặc "giảm nhiễu" tính thời vụ** để nhìn rõ xu hướng thực sự.

Làm sạch dữ liệu

- **Xử lý tính thời vụ trong dữ liệu**
  - Biểu đồ nhanh chóng cho thấy đây là dữ liệu có tính thời vụ cao, nhưng chỉ khi chúng ta lập biểu đồ đúng.





Làm sạch dữ liệu

- **Xử lý tính thời vụ trong dữ liệu**
  - Khi dữ liệu có tính mùa vụ mạnh, nó làm "nhiễu" các phân tích khác:
    - Che mất **xu hướng dài hạn** (trend).
    - Làm **bất ổn** các mô hình không mong muốn có seasonality.
    - Khi so sánh các ngày/tháng → sai lầm vì không cùng điều kiện mùa vụ (seasonally-adjusted comparison).

Làm sạch dữ liệu

➢ Xử lý tính thời vụ trong dữ liệu

- Những thao tác làm sạch dữ liệu liên quan tới tính thời vụ:
  - Tách **xu hướng + mùa vụ + phần dư**
  - Khử tính mùa vụ (deseasonalizing data)

$$Deseasonalized = \frac{X(t)}{Seasonal_index}$$

hoặc = $X(t) - Seasonal(t)$

  - Làm mượt để giảm nhiễu thời vụ ngắn hạn: dùng trung bình động theo chu kỳ mùa (ví dụ MA(12) cho chuỗi tháng)
  - Loại bỏ outlier do mùa vụ đặc biệt
    Ví dụ ngày Black Friday có số lượng bán hàng rất lớn, dễ gây lệch mô hình.

Làm sạch dữ liệu

- **Chuẩn hoá định dạng thời gian**
  - **Chuẩn hóa định dạng thời gian** là quá trình biến đổi các thông tin thời gian (timestamp, ngày tháng, giờ…) về **một định dạng thống nhất**, hợp lệ, dễ xử lý, và phù hợp với mô hình chuỗi thời gian.
  - Trong dữ liệu thực tế, dữ liệu thời gian thường **không nhất quán**, gây ra các vấn đề
    - Không thể sắp xếp đúng
    - Không thể nhóm dữ liệu (resample / downsample / upsample / rolling window) nếu nhãn thời gian không chuẩn.
    - Dữ liệu ghi nhận có thể lệch múi giờ → phân tích sai
    - Dữ liệu có thể bị thiếu hoặc thừa do lỗi format

Làm sạch dữ liệu

- **Chuẩn hoá định dạng thời gian**
  - **Chuẩn hóa định dạng thời gian** cần
    - Các định dạng này cần được chuyển về **một chuẩn duy nhất**, thường là: **YYYY-MM-DD HH:MM:SS (ISO 8601)**
    - Chuẩn hóa múi giờ (timezone normalization): Đưa tất cả về UTC, hoặc về một timezone gốc. Đây là bước cực quan trọng nhưng hay bị bỏ qua.
    - Chuẩn hóa tần suất (frequency normalization): Dữ liệu có thể không đồng đều (ví dụ ghi nhận lúc 08:01, 08:01:32, 08:02:00). Ta cần đưa về tần suất cố định (1 phút/ 5 phút/ 1 giờ/ 1 ngày)
    - Xử lý nhãn thời gian bị thiếu

Làm sạch dữ liệu

➢ Ngăn ngừa việc nhìn trước không chủ ý

- **Nhìn trước** (Look-ahead hay look-ahead bias) xảy ra khi trong **quá trình phân tích**, dữ liệu từ **tương lai** bị **vô tình sử dụng** để dự đoán **quá khứ hoặc hiện tại**.
  - Nói cách khác: mô hình "nhìn thấy trước" thông tin mà nó **không thể có trong thực tế**.
  - Đây là một dạng **data leakage** cực nguy hiểm, vì dẫn đến đánh giá quá lạc quan về hiệu suất mô hình.
- **Nhìn trước** (Look-ahead) thường gặp
  - Sử dụng dữ liệu tương lai
  - Tính chỉ số từ toàn bộ chuỗi trước khi phân tách train/test
  - Thay đổi tần suất hoặc nội suy không đúng cách

Làm sạch dữ liệu

- **Ngăn ngừa việc nhìn trước không chủ ý**
  - **Các lưu ý tránh look-ahead**
    - Chỉ dùng dữ liệu quá khứ và hiện tại để dự đoán tương lai.
    - Phân tách train/test theo thời gian, không xáo trộn (shuffle).
      - Train: thời gian trước
      - Test: thời gian sau
    - Tính các đặc trưng (feature) dạng rolling, moving average, lag dựa trên quá khứ.
    - Đảm bảo không dùng target tương lai làm feature.
    - Kiểm tra kỹ các thao tác tiền xử lý dữ liệu: resampling, interpolation phải dựa trên quá khứ.
    - Simulation phải theo thời gian thực.

Mô phỏng dữ liệu chuỗi thời gian

- Khái niệm mô phỏng dữ liệu chuỗi thời gian
- Vì sao cần mô phỏng
- Các phương pháp mô phỏng phổ biến

Mô phỏng dữ liệu chuỗi thời gian

- **Mô phỏng dữ liệu là gì?**
  - Tạo ra dữ liệu **giả lập** có đặc tính giống dữ liệu thực tế
  - Trong chuỗi thời gian: tạo ra các chuỗi có xu hướng, mùa vụ, nhiễu,…
  - Giúp kiểm thử thuật toán, huấn luyện mô hình, hoặc đánh giá hiệu năng
- **Ví dụ:**

📈 Tài chính: mô phỏng thị trường, rủi ro

Y tế: tạo dữ liệu bệnh nhân giả lập

🌐 CNTT: mô phỏng lưu lượng mạng

🚗 Ô tô tự lái: mô phỏng dữ liệu cảm biến

Mô phỏng dữ liệu chuỗi thời gian

- **Tại sao cần mô phỏng?**
  - Kiểm tra tính đúng đắn & độ ổn định của mô hình trước khi có dữ liệu thực
  - Bổ sung dữ liệu khi dữ liệu thực tế bị hạn chế hoặc nhạy cảm
  - Đào tạo mô hình học máy với dữ liệu đa dạng
  - Nghiên cứu tác động của noise, outlier, missing data
  - Thử nghiệm các chiến lược dự báo hoặc điều khiển
  - Tái hiện các tình huống hiếm gặp (ví dụ: sự cố hệ thống, khủng hoảng)

Mô phỏng dữ liệu chuỗi thời gian

- Chúng ta sẽ có thể tạo dữ liệu chuỗi thời gian thông qua mô phỏng. Nhưng việc mô phỏng **không đơn giản**, vì chuỗi thời gian có các đặc trưng phức tạp:
  - Trend (xu hướng dài hạn)
  - Seasonality (tính mùa vụ / chu kỳ)
  - Autocorrelation (liên hệ giữa các thời điểm)
  - Heteroscedasticity (biến động thay đổi theo thời gian)
  - Outlier / shock events

Mô phỏng dữ liệu chuỗi thời gian

➢ Các vấn đề phổ biến khi mô phỏng

- Không tái tạo đúng cấu trúc phụ thuộc thời gian (temporal dependence):
  - Nhiều người tạo dữ liệu ngẫu nhiên độc lập (i.i.d.) trong khi chuỗi thời gian thực có autocorrelation, tức giá trị hiện tại phụ thuộc vào quá khứ
  - Không tái tạo seasonal / cyclical pattern
- Khó tái tạo dữ liệu không đồng nhất (heteroscedastic): cần có mô hình biến động (variance) thay đổi theo thời gian
- Khó tái hiện nhiễu thực tế: Mô phỏng đơn giản thường sử dụng nhiễu dạng i.i.d Gaussian trong khi nhiễu thực tế không phải luôn Gaussian, có thể autocorrelated, heteroscedastic,… .

Mô phỏng dữ liệu chuỗi thời gian

➢ Các vấn đề phổ biến khi mô phỏng

- Xử lý outlier và sự kiện bất thường: Nhiều mô phỏng đơn giản không có outlier → mô hình sẽ không học cách phản ứng
- Vấn đề tần suất và đồng bộ hóa dữ liệu:
  - Dữ liệu thực tế có tần suất không đều: 1s, 5s, 1 phút, 1 ngày. Nếu mô phỏng cố định tần suất → không kiểm thử được các vấn đề interpolation, resampling
- Giới hạn độ dài chuỗi
  - Chuỗi quá ngắn → không thể tái hiện pattern dài hạn, seasonal dài
  - Chuỗi quá dài → khó kiểm soát, sinh dữ liệu không gần với thực tế

Mô phỏng dữ liệu chuỗi thời gian

- **Xu hướng chung trong mô phỏng chuỗi thời gian.**
  - Mô phỏng dữ liệu là một lĩnh vực khoa học dữ liệu hiếm khi được giảng dạy, nhưng lại là một kỹ năng đặc biệt hữu ích cho dữ liệu chuỗi thời gian.
    - Điều này xuất phát từ một trong những nhược điểm của việc có dữ liệu thời gian: không có hai điểm dữ liệu nào trong cùng một chuỗi thời gian có thể so sánh chính xác vì chúng xảy ra ở những thời điểm khác nhau.
    - Nếu chúng ta muốn suy nghĩ về những gì có thể đã xảy ra tại một thời điểm nhất định, chúng ta hãy chuyển sang thế giới mô phỏng.

Mô phỏng dữ liệu chuỗi thời gian

- **Xu hướng chung trong mô phỏng chuỗi thời gian.**
  - Mô phỏng có thể đơn giản hoặc phức tạp.
    - Trong các phương pháp đơn giản, chúng ta sẽ gặp dữ liệu tổng hợp theo dạng thống kê về chuỗi thời gian, chẳng hạn như dưới dạng bước ngẫu nhiên (random walk), dưới dạng tổng tích lũy của một quá trình ngẫu nhiên, hoặc bởi một hàm tuần hoàn (chẳng hạn như đường cong sin).

Mô phỏng dữ liệu chuỗi thời gian

- **Xu hướng chung trong mô phỏng chuỗi thời gian.**
  - Mô phỏng có thể đơn giản hoặc phức tạp.
    - Trong các phương pháp phức tạp hơn, nhiều nhà khoa học và kỹ sư xây dựng việc mô phỏng chuỗi thời gian bằng các quy tắc tác động lên sự phát triển của một mô hình.
      - Mô phỏng dựa trên kinh nghiệm (heuristic)
      - Mô phỏng sự kiện rời rạc
      - Mô phỏng dựa trên vật lý

      Ví dụ: Khí tượng, Tài chính, Dịch tễ học, Hóa học lượng tử, Vật lý plasma

Mô phỏng dữ liệu chuỗi thời gian

- **Các phương pháp mô phỏng**

Mô hình thống kê

- White noise
- Random walk
- AR, MA, ARIMA
- ARCH, GARCH (mô phỏng biến động)

Mô hình học sâu / sinh dữ liệu

- Generative Adversarial Networks (GANs)
- Variational Autoencoders (VAEs)

Mô phỏng dựa trên quy tắc / domain

- Dựng dữ liệu dựa trên mô hình vật lý hoặc luật vận hành

Mô phỏng dữ liệu chuỗi thời gian

- **Các phương pháp mô phỏng**
  - **Mô hình thống kê** là phương pháp truyền thống nhất để mô phỏng dữ liệu chuỗi thời gian.
    - Chúng đặc biệt hữu ích khi chúng ta biết động lực học cơ bản của một hệ thống ngẫu nhiên và muốn ước tính một vài tham số chưa biết hoặc xem các giả định khác nhau sẽ tác động như thế nào đến quá trình ước tính tham số. Ngay cả đối với các hệ thống vật lý, đôi khi mô phỏng thống kê vẫn tốt hơn.

Mô phỏng dữ liệu chuỗi thời gian

- **Các phương pháp mô phỏng**
  - **Mô hình thống kê** là phương pháp truyền thống nhất để mô phỏng dữ liệu chuỗi thời gian.
    - Mô phỏng thống kê dữ liệu chuỗi thời gian cũng khá hữu ích khi chúng ta cần có một thước đo định lượng xác định để xác định mức độ không chắc chắn của chính mình về độ chính xác của các mô phỏng. Trong các mô phỏng thống kê truyền thống, chẳng hạn như mô hình ARIMA (sẽ được thảo luận trong Chương 4), các công thức tính sai số đã được thiết lập rõ ràng, có nghĩa là để hiểu một hệ thống với một mô hình thống kê cơ bản được đặt ra, bạn không cần phải chạy nhiều mô phỏng để đưa ra các khẳng định số về sai số và phương sai.

Mô phỏng dữ liệu chuỗi thời gian

- **Các phương pháp mô phỏng**
  - **Mô phỏng học sâu**
    - Mô phỏng học sâu cho chuỗi thời gian là một lĩnh vực còn non trẻ nhưng đầy hứa hẹn.
      - Ưu điểm của học sâu là các động lực học phi tuyến tính rất phức tạp có thể được ghi lại trong dữ liệu chuỗi thời gian ngay cả khi người thực hành không hiểu đầy đủ về động lực học.
      - Tuy nhiên, đây cũng là một nhược điểm, vì người thực hành không có cơ sở nguyên tắc để hiểu động lực học của hệ thống.

Mô phỏng dữ liệu chuỗi thời gian

- **Các phương pháp mô phỏng**
  - **Mô phỏng học sâu**
    - Mô phỏng học sâu cũng hứa hẹn những ứng dụng đòi hỏi sự riêng tư.
      - Ví dụ, học sâu đã được sử dụng để tạo dữ liệu chuỗi thời gian không đồng nhất tổng hợp cho các ứng dụng y tế dựa trên dữ liệu chuỗi thời gian thực nhưng không có khả năng rò rỉ thông tin riêng tư.
      - Một tập dữ liệu như vậy, nếu thực sự có thể được tạo ra mà không có bất kỳ rò rỉ nào về quyền riêng tư, sẽ vô cùng quý giá vì các nhà nghiên cứu có thể tiếp cận một lượng lớn dữ liệu y tế (thường rất tốn kém và vi phạm quyền riêng tư).

Mô phỏng dữ liệu chuỗi thời gian

- **Các phương pháp mô phỏng**
  - **Mô phỏng học dựa trên quy tắc**
    - Mô phỏng dựa trên kinh nghiệm (heuristic): Chúng ta quyết định cách mô hình vận hành, đảm bảo nó hợp lý và mã hóa nó, từng quy tắc một.
    - Mô phỏng sự kiện rời rạc: Chúng ta xây dựng các tác nhân riêng lẻ với các quy tắc nhất định trong mô hình của mình và sau đó chạy các tác nhân đó để xem mô hình tiến hóa như thế nào theo thời gian.
    - Mô phỏng dựa trên vật lý: Chúng ta áp dụng các định luật vật lý để xem một hệ thống tiến hóa như thế nào theo thời gian.

Lưu trữ dữ liệu chuỗi thời gian

Ví dụ kiến trúc

Thu thập → Lưu trữ → Phân tích → Trực quan hóa

Mục tiêu của phần này

- Hiểu các yêu cầu khi lưu trữ dữ liệu chuỗi thời gian
- Nhận biết thách thức & giải pháp
- Các công nghệ & công cụ phổ biến

Lưu trữ dữ liệu chuỗi thời gian

- **Vì sao cần lưu trữ?**
  - Chuỗi thời gian = dữ liệu gắn với thời gian → cần **bảo toàn thứ tự & độ chính xác**
  - Đặc thù dữ liệu chuỗi thời gian
    - Khối lượng lớn & tăng liên tục
    - Viết nhiều hơn đọc
    - Chủ yếu truy vấn theo **thời gian** hoặc **cửa sổ thời gian**
    - Cần tối ưu lưu trữ & nén

Lưu trữ dữ liệu chuỗi thời gian

➢ **Thách thức lưu trữ**

- Tốc độ ghi dữ liệu cao
- Dữ liệu lỗi thời → cần xoá hoặc nén
- Đảm bảo tính toàn vẹn & độ chính xác
- Truy vấn hiệu quả theo thời gian & trên tập lớn

Ví dụ:

- Y tế: lưu tín hiệu ECG, IoT sức khoẻ
- Tài chính: giá chứng khoán, giao dịch
- CNTT: log server, giám sát mạng
- Công nghiệp: cảm biến máy móc

Lưu trữ dữ liệu chuỗi thời gian

- **Các chiến lược lưu trữ**
  - *Flat files (CSV, Parquet):* đơn giản, phù hợp dữ liệu nhỏ
  - *Cơ sở dữ liệu quan hệ (RDBMS):* MySQL, PostgreSQL
  - *Time Series Databases (TSDB):* InfluxDB, TimescaleDB, OpenTSDB, Prometheus
  - *Kho dữ liệu phân tán:* Hadoop, BigQuery

Lưu trữ dữ liệu chuỗi thời gian

- **Các chiến lược lưu trữ**
  - **Time Series Database (TSDB)**
    - Tối ưu cho dữ liệu time-series
    - Hỗ trợ:
      - Nén dữ liệu
      - Ghi nhanh
      - Truy vấn theo thời gian hiệu quả
      - Downsampling & retention policies

Lưu trữ dữ liệu chuỗi thời gian

❑ Nhận xét

- Dữ liệu chuỗi thời gian cần lưu trữ chuyên biệt
- TSDB mang lại hiệu quả cao hơn RDBMS cho dữ liệu lớn
- Chọn công cụ & chiến lược phù hợp với nhu cầu
  - Lựa chọn độ phân giải phù hợp (giây, phút, giờ)
  - Đặt quy tắc duy trì dữ liệu (**retention policy** - giữ dữ liệu bao lâu)
  - Sử dụng nén & downsampling
  - Lập chỉ mục trên trường thời gian

Trực quan hóa dữ liệu chuỗi thời gian

- Vai trò của trực quan hóa dữ liệu chuỗi thời gian
- Các loại biểu đồ thường dùng
- Một số công cụ & thực tiễn tốt

📌 Đặc thù khi trực quan hóa chuỗi thời gian

- Dữ liệu có thứ tự thời gian → trục X thường là thời gian
- Có thể nhiều chiều (nhiều chuỗi, nhiều biến)
- Có thể rất dài → cần tổng hợp, phóng to/thu nhỏ

Trực quan hóa dữ liệu chuỗi thời gian

➢ Vai trò của trực quan hóa dữ liệu chuỗi thời gian

- Trực quan hóa dữ liệu chuỗi thời gian là quá trình **biến các điểm dữ liệu theo thời gian thành biểu đồ** nhằm giúp hiểu cấu trúc và hành vi của dữ liệu trước khi mô hình hóa.
- Ví dụ:
  - Y tế: theo dõi tín hiệu sinh học
  - Tài chính: xu hướng giá cổ phiếu
  - IT: giám sát server, lưu lượng mạng
  - Công nghiệp: theo dõi máy móc

Trực quan hóa dữ liệu chuỗi thời gian

➢ Vai trò của trực quan hóa dữ liệu chuỗi thời gian

Trực quan hóa là bước quan trọng để hiểu & truyền đạt dữ liệu, nhằm giúp:

- Nhận diện nhanh xu hướng & mẫu
- Phát hiện bất thường, mùa vụ, chu kỳ
- Dễ truyền đạt thông tin cho người khác
- Hỗ trợ phân tích & ra quyết định
- Đánh giá mô hình dự báo

→Cần hiểu mục tiêu & dữ liệu để chọn cách trực quan hóa phù hợp

Trực quan hóa dữ liệu chuỗi thời gian

- **Các loại biểu đồ cơ bản**

Biểu đồ đường (Plotting/Line plot)

- Phổ biến nhất
- Hiển thị giá trị theo thời gian

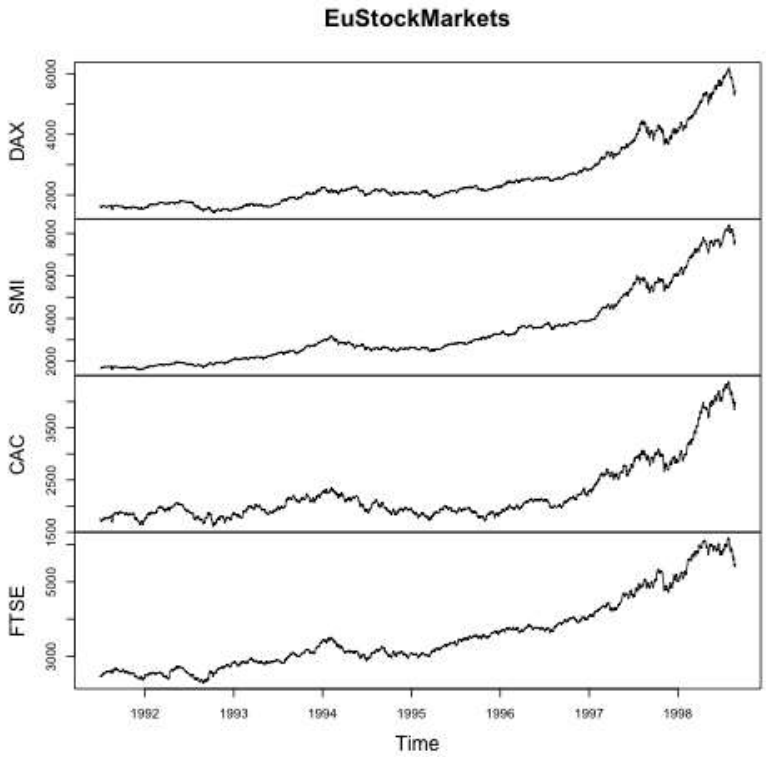
Biểu đồ tần suất của giá trị (Histogram)

- Tốt cho dữ liệu theo khoảng thời gian rõ rệt (ngày, tuần, tháng)

- Biểu đồ điểm/phân tán (Scatter Plot)
  - Phân tán giá trị theo thời gian
  - Biểu đồ phân tán trễ (Lag scatter plot) (quan trọng hơn trong time series)

Trực quan hóa dữ liệu chuỗi thời gian

- **Các loại biểu đồ cơ bản**
  - Biểu đồ đường (Plotting/Line plot)
    - Đây là biểu đồ phổ biến nhất trong phân tích chuỗi thời gian.
      - Trục X = thời gian
      - Trục Y = giá trị của chuỗi
    - Luôn nên thử đầu tiên khi nhận được dữ liệu choỗi thời gian để kiểm tra chất lượng thô và quan sát hành vi tổng thể



Trực quan hóa dữ liệu chuỗi thời gian

- **Các loại biểu đồ cơ bản**
  - Biểu đồ đường (Plotting/Line plot)
    - giúp bạn nhìn thấy:
      - Xu hướng (trend): tăng dần/ giảm dần/ đi ngang
      - Mùa vụ (seasonality): lặp lại theo ngày, tuần, tháng, năm. Ví dụ: nhiệt độ mùa hè cao, mùa đông thấp
      - Biến động (volatility): các đoạn dốc mạnh = biến động cao, các đoạn ngang = ổn định
      - Outliers (điểm bất thường): các điểm nhảy lên hoặc rơi xuống bất ngờ
      - Change point (đột biến cấu trúc): đường cong đổi hướng đột ngột do một sự kiện

Trực quan hóa dữ liệu chuỗi thời gian

- **Các loại biểu đồ cơ bản**
  - Biểu đồ đường (Plotting/Line plot)
    - Bước đầu tiên và quan trọng nhất trong bất kỳ phân tích chuỗi thời gian nào là vẽ biểu đồ các quan sát theo thời gian. Tuy nhiên, việc vẽ biểu đồ chuỗi thời gian không dễ như bạn nghĩ.
      - Việc lựa chọn thang đo, kích thước của điểm cắt và cách các điểm được vẽ (ví dụ: dưới dạng một đường liên tục hoặc các điểm chấm hoặc dấu chéo riêng biệt) có thể ảnh hưởng đáng kể hình thức của biểu đồ, vì vậy nhà phân tích phải hết sức cẩn thận và phán đoán.
      - Việc biểu diễn dữ liệu có thể gợi ý rằng việc cân nhắc chuyển đổi chúng là hợp lý, ví dụ, bằng cách lấy logarit hoặc căn bậc hai.

Trực quan hóa dữ liệu chuỗi thời gian

- **Các loại biểu đồ cơ bản**
  - Biểu đồ đường (Plotting/Line plot)
    - Nhiều chuỗi thời gian **không phù hợp trực tiếp** cho mô hình do:
      - phương sai thay đổi theo thời gian (**heteroscedasticity**)
      - giá trị tăng theo cấp số nhân
      - phân phối lệch (skewed)
      - độ lớn khác biệt quá nhiều
      - outlier gây méo mô hình
      - không stationary
    - Hai biến đổi **log** và **sqrt** là những kỹ thuật cơ bản nhất để xử lý các vấn đề này

Trực quan hóa dữ liệu chuỗi thời gian

- **Các loại biểu đồ cơ bản**
  - Biểu đồ đường (Plotting/Line plot)
    - Biến đổi **log** giúp dễ trực quan dữ liệu hơn
      - Ổn định phương sai: Nhiều chuỗi có biên độ dao động lớn, Log giảm độ dao động của các giá trị lớn.
      - Chuyển tăng trưởng theo cấp số nhân → tuyến tính

$$x_t = Ae^{bt} \rightarrow \log x_t = \log A + bt$$

      - Chuyển nhân thành cộng

$$x_t = trend(t) \cdot season(t) \cdot noise(t)$$

$$\rightarrow \log x_t = \log trend + \log season + \log noise$$

      - Giảm độ lệch phân phối: log sẽ làm phân phối gần Gaussian hơn
      - Giảm ảnh hưởng outlier: log "nén" giá trị lại.

Trực quan hóa dữ liệu chuỗi thời gian

- **Các loại biểu đồ cơ bản**
  - Biểu đồ đường (Plotting/Line plot)
    - Biến đổi **sqrt** (sqrt(x) = $\sqrt{x}$) cũng giúp nén giá trị dữ liệu lại nhưng ở mức độ nhẹ hơn log.

      x = [1, 4, 9, 16]

      sqrt(x) = [1, 2, 3, 4]

Trực quan hóa dữ liệu chuỗi thời gian

- **Các loại biểu đồ cơ bản**
  - Biểu đồ đường (Plotting/Line plot)
    - Biến đổi **Box-Cox:** Họ biến đổi này phổ biến trong lĩnh vực thiết kế thí nghiệm, có dạng sau:

$$g(x) = \begin{cases} \dfrac{x^{\lambda} - 1}{\lambda} & \lambda \neq 0 \\ \ln(x) & \lambda = 0 \end{cases}$$

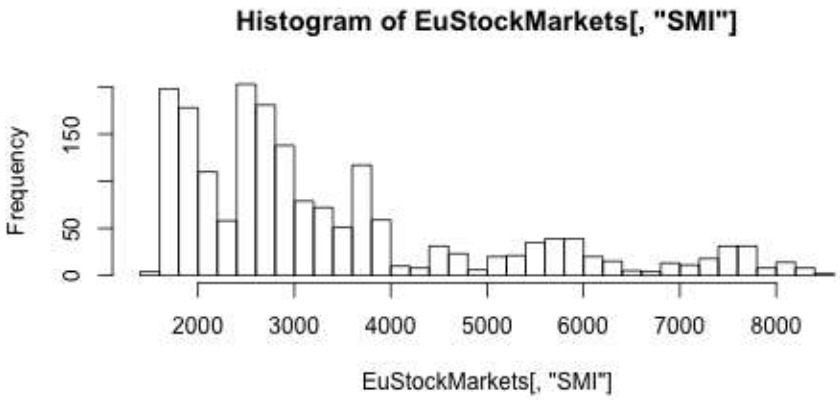
- Lưu ý: Trừ 1 và chia cho λ, hàm $g(x)$ biến thiên mượt mà khi λ tiến tới 0. Khi λ⟶0 thì (x^λ−1)/λ ⟶ln(x).
- Việc chọn λ = 0,5: phép biến đổi căn bậc hai, điều này hữu ích nếu dữ liệu là dữ liệu đếm tuân theo phân phối Poisson,
- λ = −1: là nghịch đảo của dữ liệu.

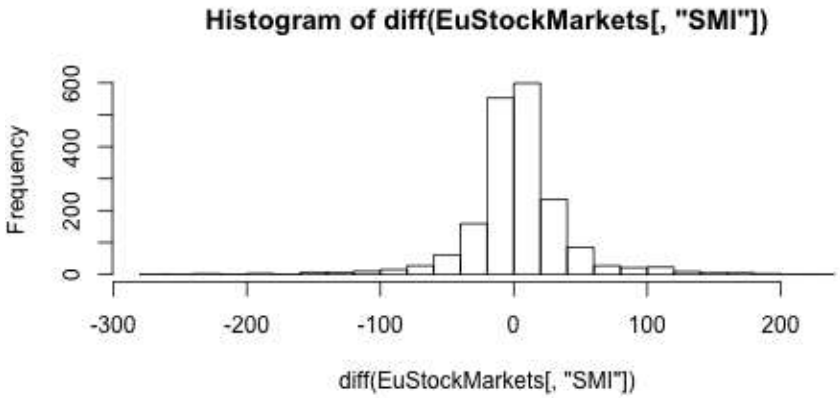
Trực quan hóa dữ liệu chuỗi thời gian

- **Các loại biểu đồ cơ bản**

  Biểu đồ tần suất của giá trị (Histogram)

  Histogram **không** quan tâm đến thời gian. Nó chỉ quan tâm đến **phân phối của giá trị trong chuỗi**.





Trực quan hóa dữ liệu chuỗi thời gian

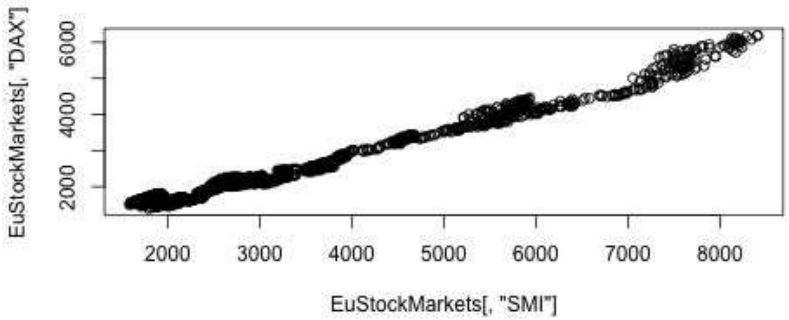
- **Các loại biểu đồ cơ bản**

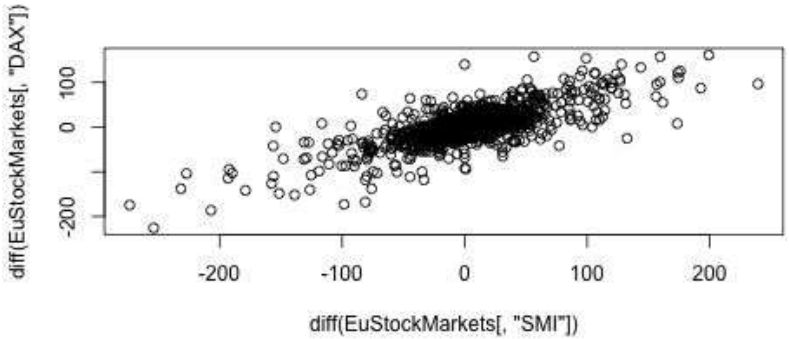
Biểu đồ tần suất của giá trị (Histogram)

- Histogram giúp bạn kiểm tra cấu trúc thống kê:
  - **Phân phối dữ liệu (normal, skewed?)**
    - ✓ Nếu gần chuẩn (Gaussian) → thuận lợi cho ARIMA
    - ✓ Nếu lệch (skewed) → có thể cần log-transform
  - **Mức độ biến động**
    - ✓ histogram rộng → variance lớn
    - ✓ histogram hẹp → variance nhỏ
  - **Phát hiện outliers:** các thanh rìa nhỏ lẻ → giá trị bất thường
  - **Kiểm tra stationarity:** Nếu theo thời gian, giá trị trung bình thay đổi → histogram từng phân đoạn sẽ khác nhau.

Trực quan hóa dữ liệu chuỗi thời gian

- **Các loại biểu đồ cơ bản**
  - Biểu đồ điểm/phân tán (Scatter Plot)
    - Phân tán giá trị theo thời gian: Giống line plot nhưng **không có đường nối**. Phương pháp truyền thống sử dụng biểu đồ phân tán cho dữ liệu chuỗi thời gian giống như các loại dữ liệu khác.

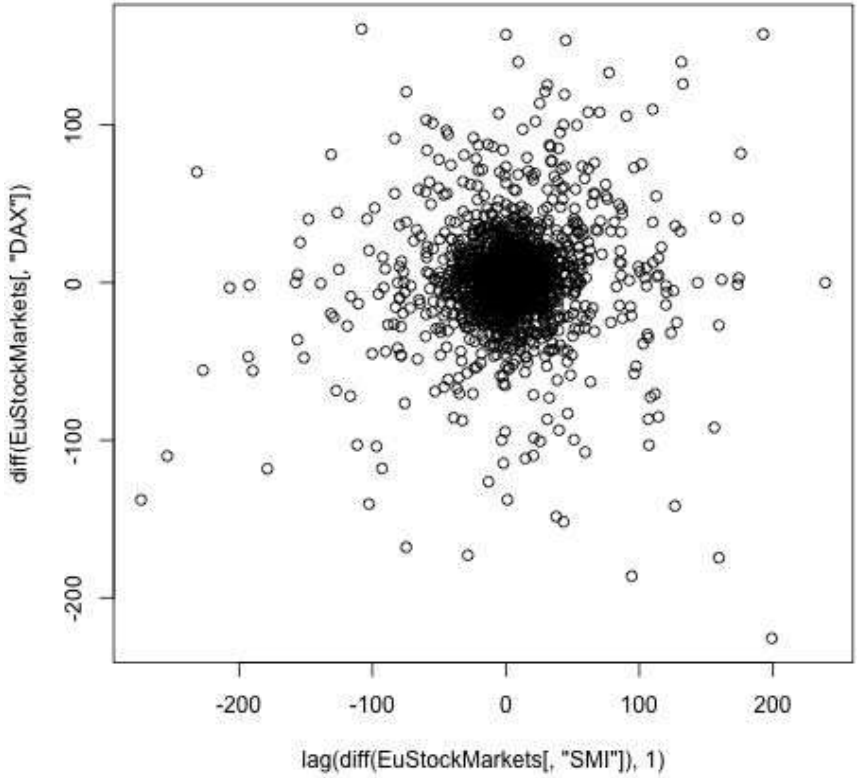




Trực quan hóa dữ liệu chuỗi thời gian

- **Các loại biểu đồ cơ bản**
  - Biểu đồ điểm (Scatter Plot)
    - Biểu đồ phân tán trễ (Lag scatter plot)
      - Trục X = giá trị tại t
      - Trục Y = giá trị trễ (t-k)

      Đây là cách kiểm tra **tính tự tương quan (autocorrelation)**.



Trực quan hóa dữ liệu chuỗi thời gian

- **Các loại biểu đồ cơ bản**
  - Biểu đồ điểm (Scatter Plot)
    - Biểu đồ phân tán trễ (Lag scatter plot)
      - Đây là cách kiểm tra **tính tự tương quan (autocorrelation)**.
        - ✓ Nếu điểm nằm gần đường chéo → có autocorrelation mạnh
        - ✓ Nếu điểm phân tán khắp nơi → gần như noise trắng
        - ✓ Nếu tạo hình ellipse nghiêng → giống AR(1)
        - ✓ Nếu tạo hình vòng → có tính chu kỳ mạnh

Trực quan hóa dữ liệu chuỗi thời gian

- **Một số công cụ hỗ trợ**

  🛠 Python: matplotlib, seaborn, plotly, bokeh, altair

  🛠 Web/Dashboard: Grafana, Kibana, Tableau, Power BI

  →<u>Lưu ý</u>:

  - Chọn độ phân giải phù hợp.
  - Sử dụng màu sắc nhất quán & có ý nghĩa.
  - Thêm nhãn, chú thích rõ ràng.
  - Tránh quá tải thông tin → giữ đơn giản